

Lab- Chapter 2

Prateek Minhas

2026-05-23

Lab work: Chapter 2 Statistical Learning

##Basic functions

```
#concatenate  
x <- c(1,3,4,5)  
x
```

```
## [1] 1 3 4 5
```

```
y = c(1,2,3,4)  
y
```

```
## [1] 1 2 3 4
```

```
#length function  
length(x)
```

```
## [1] 4
```

```
length(y)
```

```
## [1] 4
```

```
# 2 arrays will be summed up only when their lengths are same  
x+y
```

```
## [1] 2 5 7 9
```

```
class(x) #numeric means ki concatenate creates a numeric vector
```

```
## [1] "numeric"
```

```
#ls() function tells us that how many variables are there currently in our R file  
ls()
```

```
## [1] "x" "y"
```

```
#rm() - remove objects  
rm(x)#single variable delete  
ls()
```

```
## [1] "y"
```

```
rm(list=ls())#delete all variables at once  
ls()
```

```
## character(0)
```

```
#matrix() how to make a matrix - rows and columns data structure  
#data - numbers to be filled in the matrix--  
#column wise fill  
x = matrix(data = c(1,2,3,4),nrow = 2, ncol = 2)  
x
```

```
##      [,1] [,2]  
## [1,]    1    3  
## [2,]    2    4
```

```
#another way to write that-  
matrix(c(4,3,2,1),2,2)
```

```
##      [,1] [,2]  
## [1,]    4    2  
## [2,]    3    1
```

```
#if you want that rows be filled first by default  
matrix(c(1,2,3,4),2,2,byrow=TRUE)
```

```
##      [,1] [,2]  
## [1,]    1    2  
## [2,]    3    4
```

```
##? operator in front of any function , will open the description window  
?sqrt
```

```
## starting httpd help server ... done
```

```
#sqrt of a matrix , every single element gets sqrooted  
x
```

```
##      [,1] [,2]  
## [1,]    1    3  
## [2,]    2    4
```

```
sqrt(x)
```

```
##           [,1]      [,2]
## [1,] 1.000000 1.732051
## [2,] 1.414214 2.000000
```

```
#rnorm() Random number generator
```

```
x = rnorm(50) # will generate any random 50 numbers with mean 0 and sd =1
y= x + rnorm(50, mean=50, sd =0.1)
y
```

```
## [1] 49.35168 48.58682 51.24437 50.46788 50.49954 50.87297 49.54302 51.26052
## [9] 50.84564 50.50152 51.45961 50.33763 50.33663 50.47429 49.50738 49.57398
## [17] 48.29368 51.27522 47.15887 48.85566 51.68827 48.66046 49.96809 50.12883
## [25] 50.86467 49.19734 50.20305 50.55131 51.29095 51.31237 48.95534 50.09350
## [33] 49.12540 52.38440 49.61353 48.70645 49.85680 50.01343 50.59853 48.68920
## [41] 51.77553 48.80850 51.10673 51.30775 50.10073 50.43629 49.67397 50.17971
## [49] 47.75045 49.85774
```

```
# set.seed() to lock the random numbers- will produce the same numbers in every single run anywhere
#best for reproducing results.
```

```
set.seed(1303)
rnorm(50)
```

```
## [1] -1.1439763145 1.3421293656 2.1853904757 0.5363925179 0.0631929665
## [6] 0.5022344825 -0.0004167247 0.5658198405 -0.5725226890 -1.1102250073
## [11] -0.0486871234 -0.6956562176 0.8289174803 0.2066528551 -0.2356745091
## [16] -0.5563104914 -0.3647543571 0.8623550343 -0.6307715354 0.3136021252
## [21] -0.9314953177 0.8238676185 0.5233707021 0.7069214120 0.4202043256
## [26] -0.2690521547 -1.5103172999 -0.6902124766 -0.1434719524 -1.0135274099
## [31] 1.5732737361 0.0127465055 0.8726470499 0.4220661905 -0.0188157917
## [36] 2.6157489689 -0.6931401748 -0.2663217810 -0.7206364412 1.3677342065
## [41] 0.2640073322 0.6321868074 -1.3306509858 0.0268888182 1.0406363208
## [46] 1.3120237985 -0.0300020767 -0.2500257125 0.0234144857 1.6598706557
```

```
set.seed(3)
y= rnorm(100)
mean(y)
```

```
## [1] 0.01103557
```

```
var(y)
```

```
## [1] 0.7328675
```

```
sqrt(var(y))
```

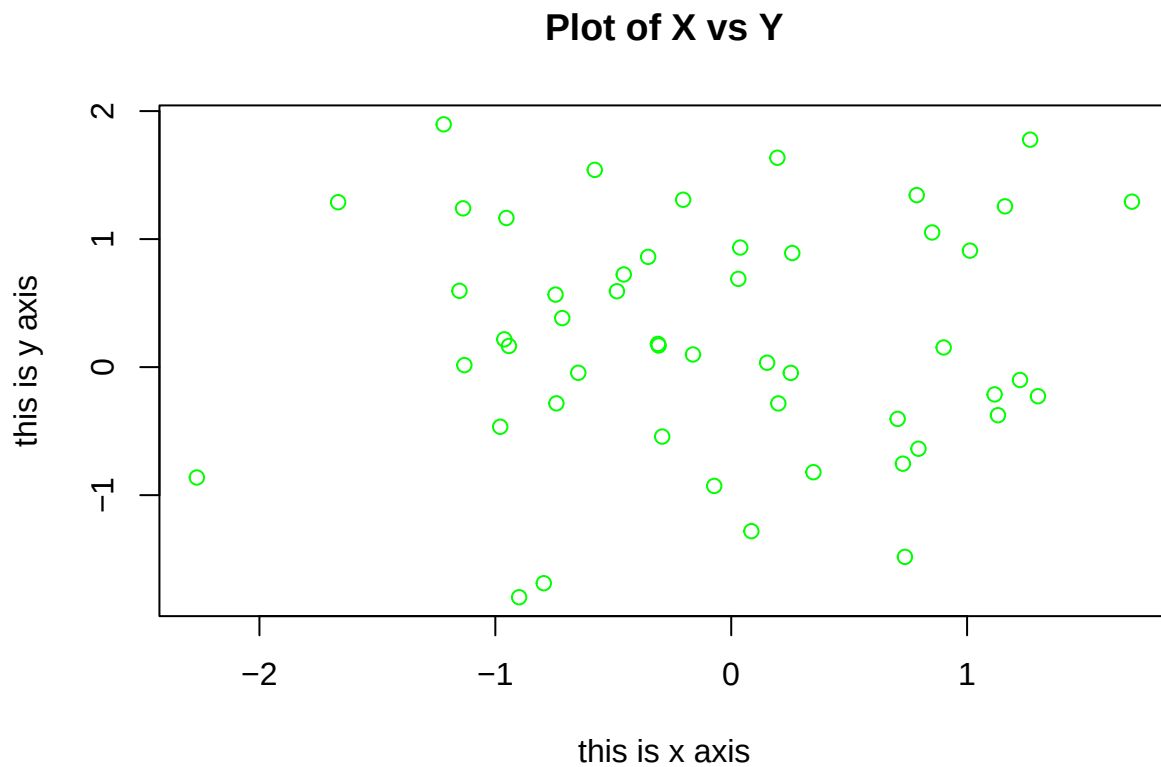
```
## [1] 0.8560768
```

```
sd(y)
```

```
## [1] 0.8560768
```

```
##Graphics
```

```
#plot() - primary way to make function, by default it makes a scatterplot  
set.seed(3)  
x=rnorm(50)  
set.seed(4)  
y=rnorm(50)  
plot(x,y)  
#creating labels to make graph more professional and adding color col  
plot(x,y,xlab="this is x axis", ylab="this is y axis", main="Plot of X vs Y",col = "green")
```



```
#seq() sequence generator  
seq(1,10)
```

```
## [1] 1 2 3 4 5 6 7 8 9 10
```

```
x=1:10 # same method different way  
x
```

```
## [1] 1 2 3 4 5 6 7 8 9 10
```

```
#length feature- will generate seq from 0 to 1 of equal gaps
```

```
x= seq(0,1,length=10)
```

```
x
```

```
## [1] 0.0000000 0.1111111 0.2222222 0.3333333 0.4444444 0.5555556 0.6666667
```

```
## [8] 0.7777778 0.8888889 1.0000000
```

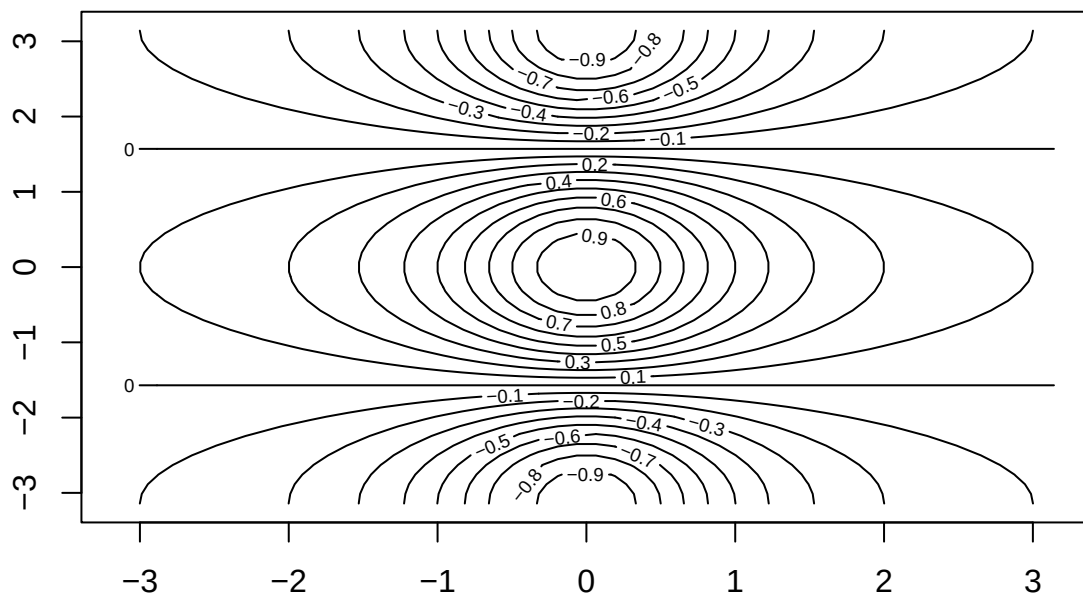
```
x=seq(-pi,pi, length=50)
```

```
#contour()- topographical maps for 3d data
```

```
y=x
```

```
z= outer(x,y,function(x,y) cos(y)/(1+x^2))
```

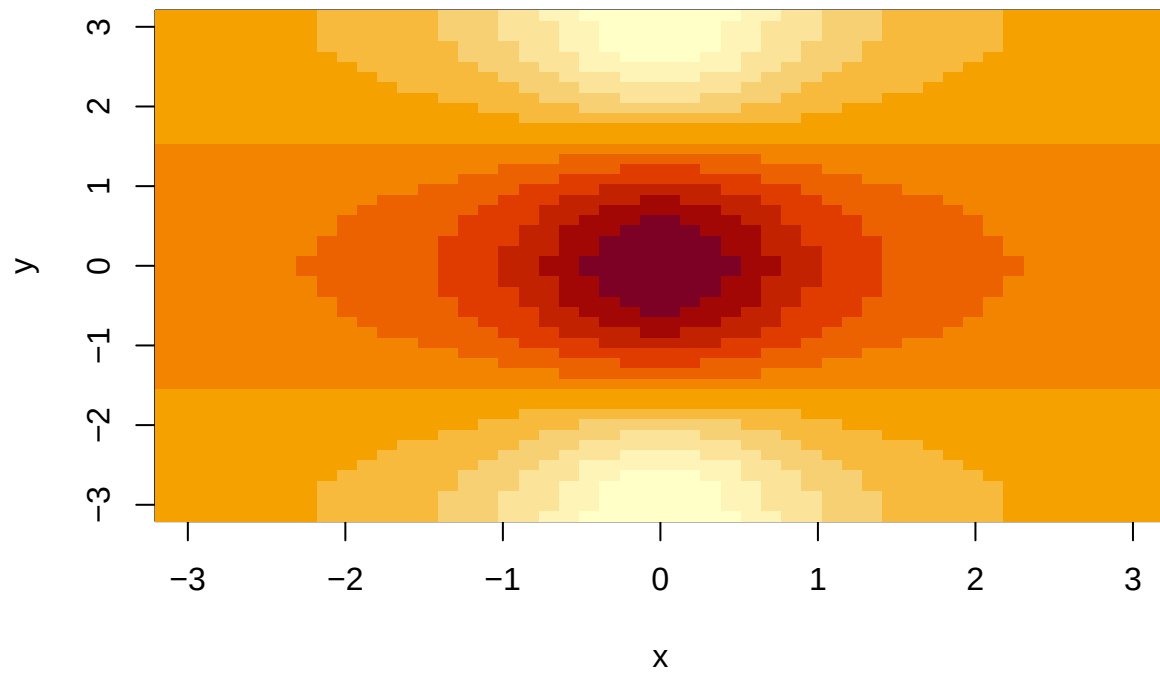
```
contour(x,y,z, nlevels = 15)
```



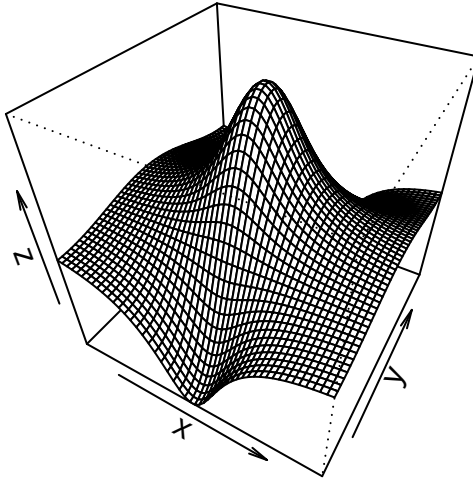
```
#image()- Heatmap
```

```
# this will put the Z matrix wherever Z is high the color will deepn , this is used for temperature.
```

```
image(x,y,z)
```



```
#persp() if we want to see 3d  
persp(x,y,z,theta = 30, phi = 40) # theta, phi used for changing camera angle
```



##Indexing

```
A <- matrix(1:16, 4, 4)
A
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    5    9   13
## [2,]    2    6   10   14
## [3,]    3    7   11   15
## [4,]    4    8   12   16
```

```
A[2, 3] #selecting row 2 column 3 that is element 10
```

```
## [1] 10
```

```
A[c(1, 3), c(2, 4)] #selecting row 1 and 3 and column 2 and 4 we get is sub matrix
```

```
##      [,1] [,2]
## [1,]    5   13
## [2,]    7   15
```

```
A[1:3, 2:4] # select all rows from 1 to 3 and all columns from 2 to 4
```

```
##      [,1] [,2] [,3]
## [1,]    5    9   13
## [2,]    6   10   14
## [3,]    7   11   15
```

```
A[1, ] #selecting only 1 row
```

```
## [1]  1  5  9 13
```

```
A[,2:3] #selecting column 2 and 3
```

```
##      [,1] [,2]
## [1,]    5    9
## [2,]    6   10
## [3,]    7   11
## [4,]    8   12
```

```
#dim() function outputs the number of rows followed by the number of dim() columns of a given matrix.
dim(A)
```

```
## [1] 4 4
```

```
##Loading Data
```

```
#read.table() function, used to import and text based data files
#the table like structure is called dataframe
Auto = read.table("Auto.data", header = TRUE) #header se v1 v2 nahi aenge and actual names aenge
#View(Auto) #to view in spreadsheet view
head(Auto) #to see only a few rows and columns
```

```
##   mpg cylinders displacement horsepower weight acceleration year origin
## 1  18         8          307        130.0   3504          12.0    70      1
## 2  15         8          350        165.0   3693          11.5    70      1
## 3  18         8          318        150.0   3436          11.0    70      1
## 4  16         8          304        150.0   3433          12.0    70      1
## 5  17         8          302        140.0   3449          10.5    70      1
## 6  15         8          429        198.0   4341          10.0    70      1
##                                name
## 1 chevrolet chevelle malibu
## 2          buick skylark 320
## 3          plymouth satellite
## 4              amc rebel sst
## 5              ford torino
## 6              ford galaxie 500
```

```
#dim() like matrix it also checks dimensions , rows and columns of the DataFrame
dim(Auto)
```

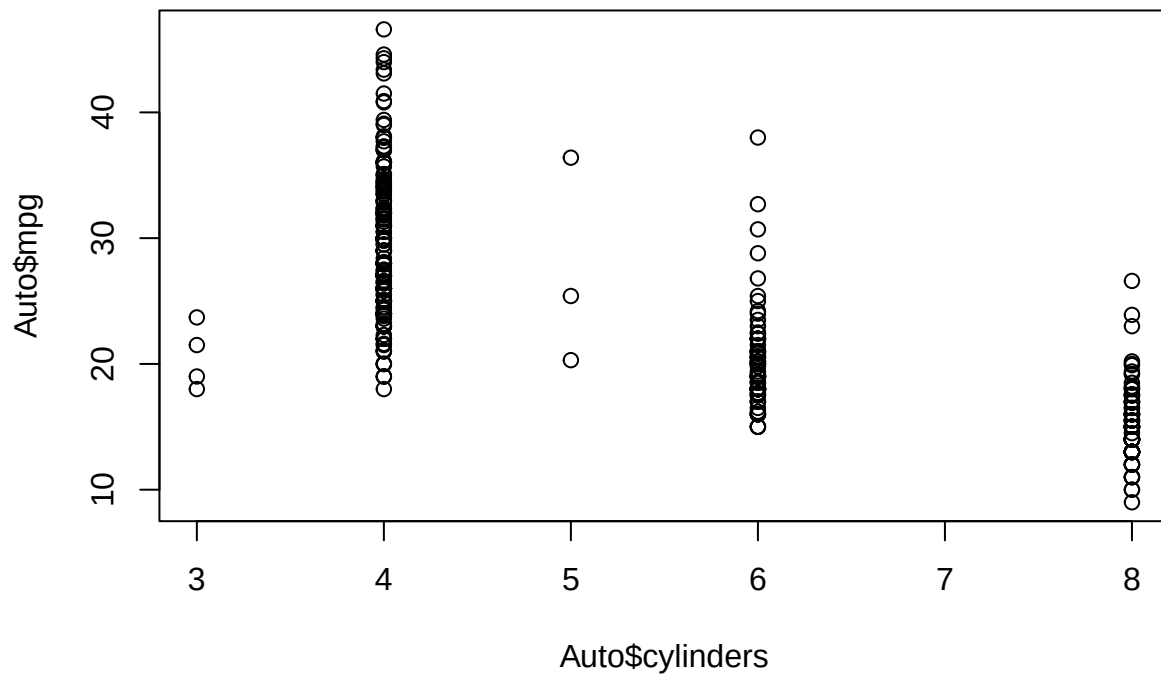
```
## [1] 397  9
```



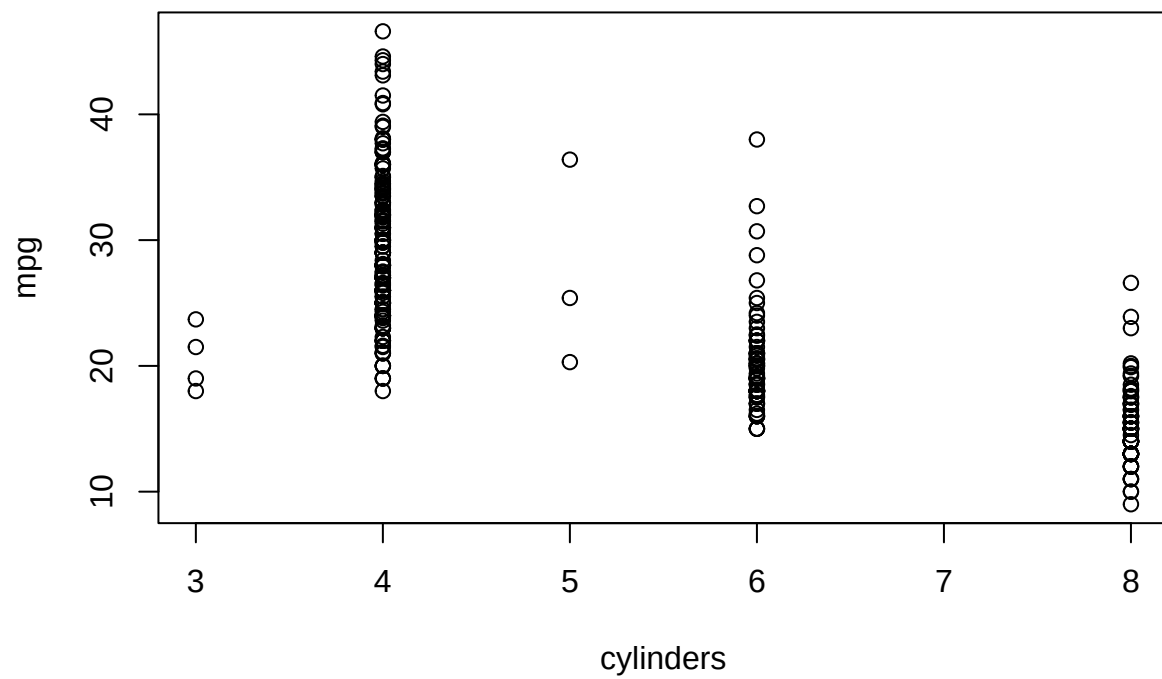
```
#na.omit(Auto) #this will remove all the rows where there is na ,meaning a data variable missing  
names(Auto)
```

```
## [1] "mpg"          "cylinders"     "displacement" "horsepower"   "weight"  
## [6] "acceleration" "year"          "origin"        "name"
```

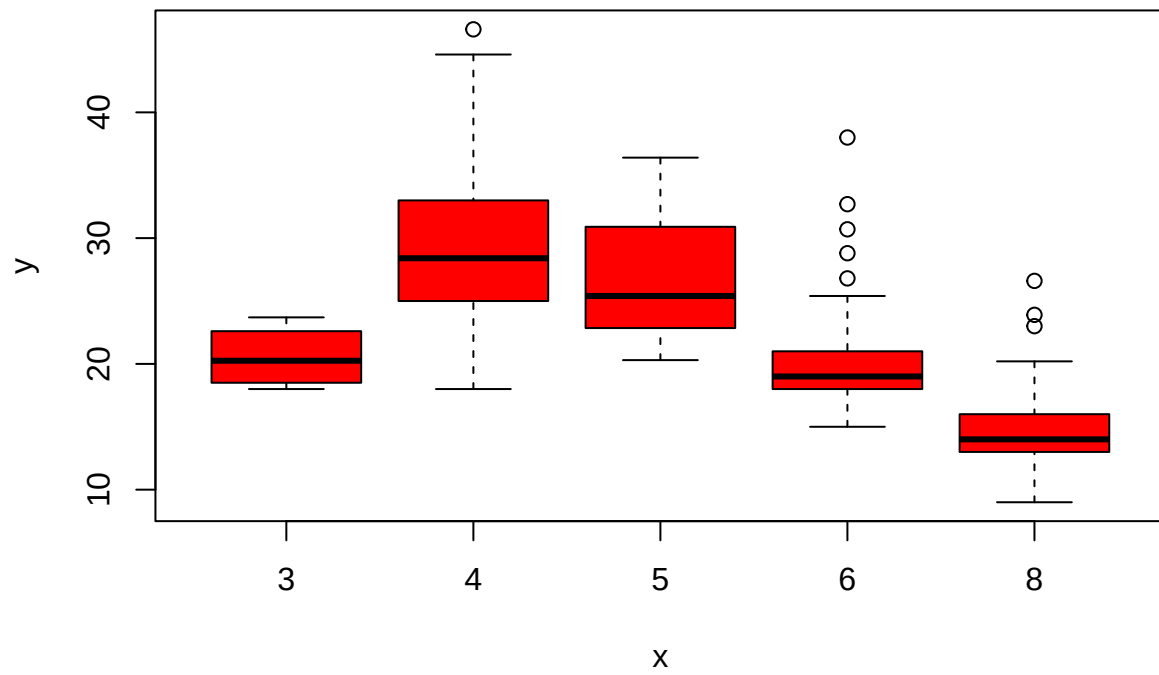
```
plot(Auto$cylinders, Auto$mpg)
```



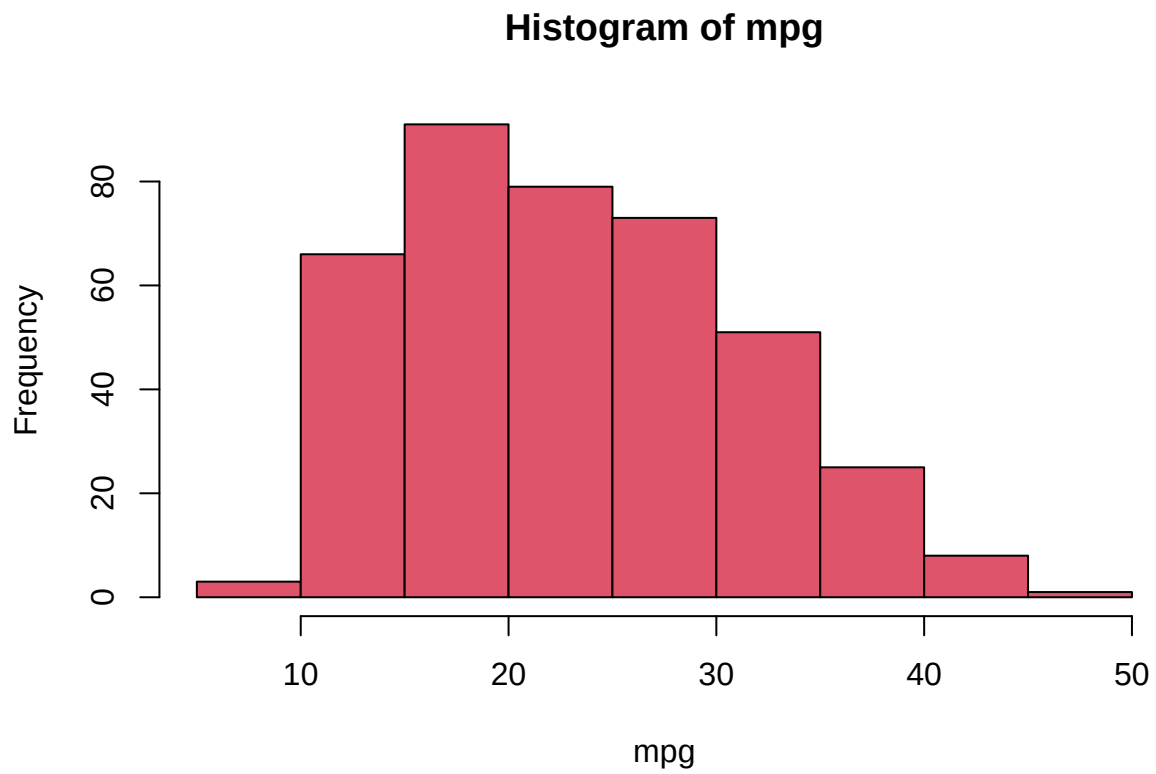
```
#we use dollar sign aur dataframe ka naam as a reference to plot the graphs  
attach(Auto)  
plot(cylinders,mpg)
```



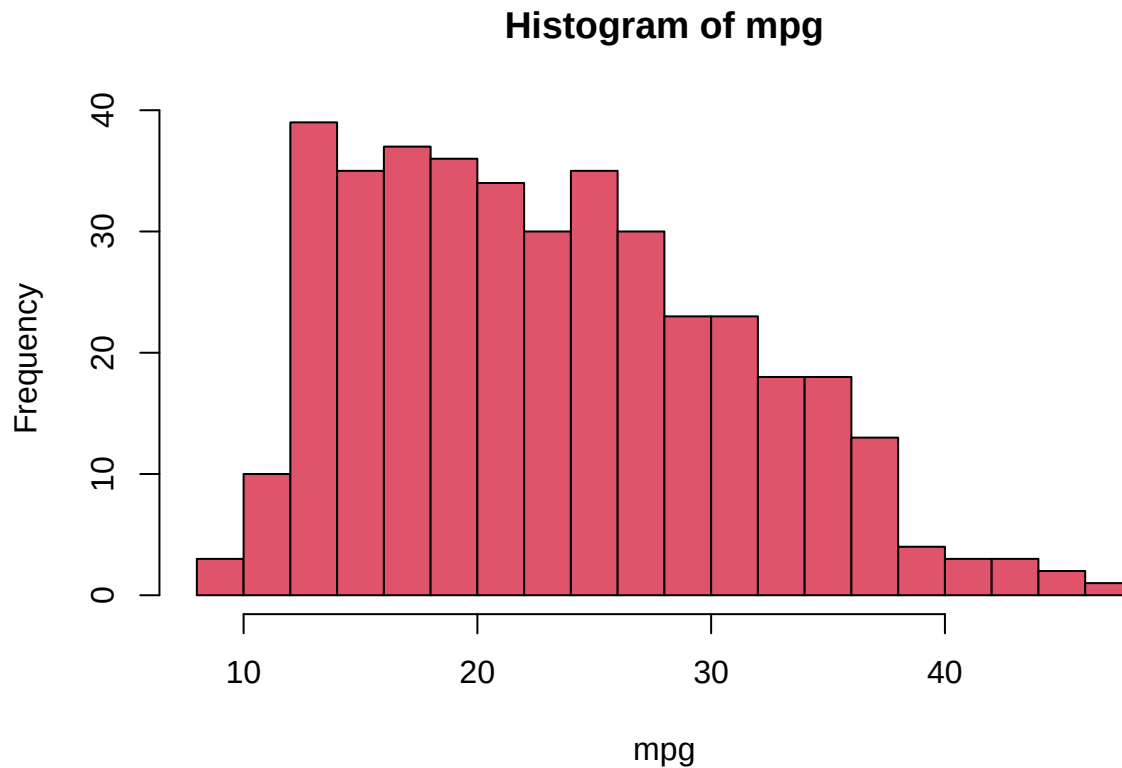
```
#as.factor() is used to convert a numeric value to a category for boxplot  
cylinders=as.factor(cylinders)  
plot(cylinders,mpg, col="red")
```



```
#hist() for histogram, used to see the shape of the vector data.  
hist(mpg, col=2)
```



```
hist(mpg, col=2 ,breaks = 15)
```



```
#pairs() for scatterplot matrix and only accepts numeric values
str(Auto[, c("mpg", "displacement", "horsepower", "weight", "acceleration")])
```

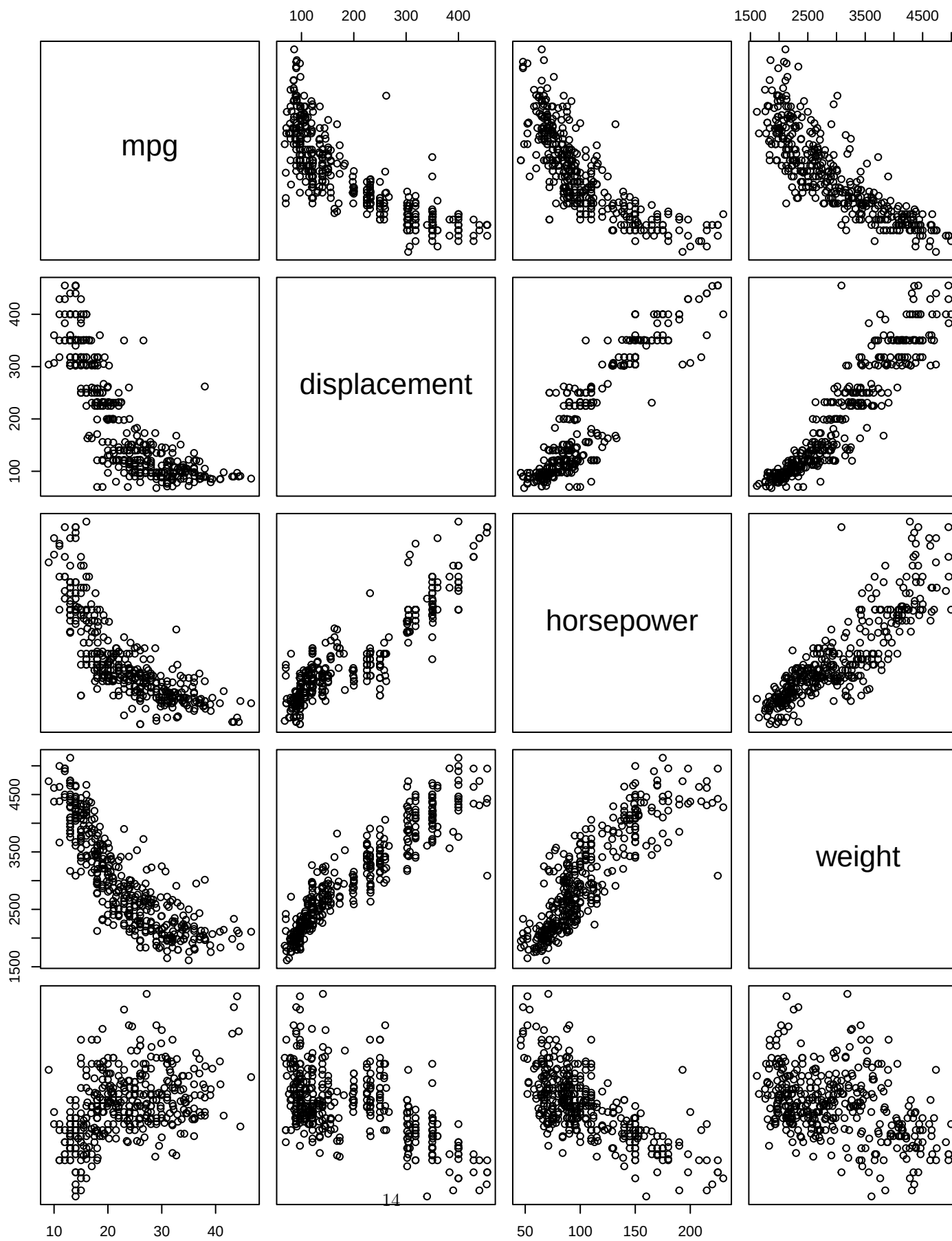
```
## 'data.frame': 397 obs. of 5 variables:
## $ mpg : num 18 15 18 16 17 15 14 14 14 15 ...
## $ displacement: num 307 350 318 304 302 429 454 440 455 390 ...
## $ horsepower : chr "130.0" "165.0" "150.0" "150.0" ...
## $ weight : num 3504 3693 3436 3433 3449 ...
## $ acceleration: num 12 11.5 11 12 10.5 10 9 8.5 10 8.5 ...
```

```
#here horse power is a charachter and wont be accepted in pairs function
```

```
#change figure
Auto$horsepower=as.numeric(Auto$horsepower)
```

```
## Warning: NAs introduced by coercion
```

```
pairs(~ mpg + displacement + horsepower + weight + acceleration, data = Auto)
```

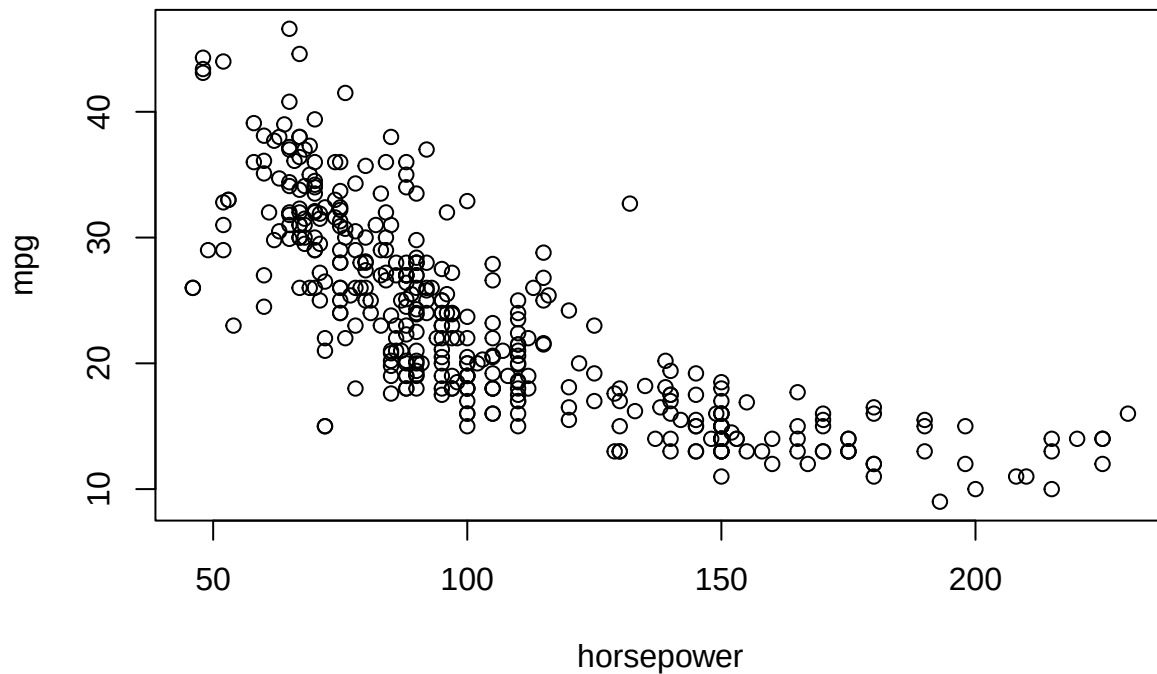


```
#windows()
plot(horsepower, mpg)
```

```
## Warning in xy.coords(x, y, xlabel, ylabel, log): NAs introduced by coercion
```

```
identify(horsepower , mpg, name)# is used to select a point and see its coordinates , used mostly for o
```

```
## Warning in xy.coords(x, y, setLab = FALSE): NAs introduced by coercion
```



```
## integer(0)
```

```
#summary() numerical overview very important
```

```
summary(Auto)
```

```
##      mpg      cylinders  displacement  horsepower      weight
##  Min.   : 9.00   Min.   :3.000   Min.   : 68.0   Min.   : 46.0   Min.   :1613
##  1st Qu.:17.50   1st Qu.:4.000   1st Qu.:104.0   1st Qu.: 75.0   1st Qu.:2223
##  Median :23.00   Median :4.000   Median :146.0   Median : 93.5   Median :2800
##  Mean   :23.52   Mean   :5.458   Mean   :193.5   Mean   :104.5   Mean   :2970
##  3rd Qu.:29.00   3rd Qu.:8.000   3rd Qu.:262.0   3rd Qu.:126.0   3rd Qu.:3609
##  Max.   :46.60   Max.   :8.000   Max.   :455.0   Max.   :230.0   Max.   :5140
```

```

##
##   acceleration      year      origin      NAs      :5
##   name
##   Min.      : 8.00   Min.      :70.00   Min.      :1.000   Length    :397
##   1st Qu.:13.80   1st Qu.:73.00   1st Qu.:1.000   N.unique  :304
##   Median :15.50   Median :76.00   Median :1.000   N.blank   : 0
##   Mean    :15.56   Mean    :75.99   Mean    :1.574   Min.nchar: 6
##   3rd Qu.:17.10   3rd Qu.:79.00   3rd Qu.:2.000   Max.nchar:36
##   Max.     :24.80   Max.     :82.00   Max.     :3.000
##

```